# Leitura — Estado, dados reais e testes: ViewModel, Flow, camadas, Koin, Ktor Client e DataStore (Sprint 2)

Guia de apoio para as duas aulas da Sprint 2: corrotinas, Flow e ViewModel (28/10, passos 11 e 12) e camadas, Koin, Ktor Client, cache local e testes de ViewModel (em vídeo, passos 13 a 16). Na Sprint 1, o app de Tarefas ganhou telas, navegação, layout adaptativo e acessibilidade, com o estado guardado na própria tela. Nesta, o estado vai para um ViewModel, as tarefas passam a vir da API de tarefas, e o app abre mesmo sem rede.

Exemplo de referência: `exemplos/tarefas-compose/`, passos 11 a 16 do `PASSOS.md`. Os slides são `slides/sistemas-moveis-slides-06.md` e `slides/sistemas-moveis-slides-07.md`. No branch `aula`, as tags `passo-11` a `passo-16` guardam o estado de cada passo. A API usada é uma API de tarefas já existente, que sobe com `docker compose up --build`.

Versões novas: Lifecycle ViewModel 2.9.6, Koin 4.2.2, Ktor Client 3.5.2, DataStore 1.2.1, kotlinx-coroutines 1.11.0 e Turbine 1.2.1. Tudo o que está descrito foi executado: os 25 testes no alvo desktop, o APK de debug, e o app no desktop contra a API no ar e, depois, com a API parada.

Capítulos:


1. A Sprint 2 e o que muda no app
2. Corrotinas para a interface
3. `StateFlow` e o ViewModel
4. Os estados da tela como classe selada
5. Camadas e injeção de dependência com Koin
6. Dados reais com o Ktor Client
7. Abrir sem rede: cache com DataStore
8. Testes de ViewModel com Turbine
9. Exercícios e dúvidas frequentes


## 1. A Sprint 2 e o que muda no app

| Critério | Peso | Capítulos |
|---|---|---|
| Estado e arquitetura | 25% | 3 e 5 |
| Estados da interface | 15% | 4 |
| Dados reais e erro de rede | 25% | 6 |

| Critério | Peso | Capítulos |
|---|---|---|
| Persistência local | 10% | 7 |
| Testes (interface e ViewModel) | 15% | 8 |
| App e documentação | 10% | — |

O app ao fim da sprint:

```
  Telas (@Composable)  ◄── estado ──  ViewModel  ──►  Repositório com cache  ──►  a API de
tarefas
          └──────── eventos ──────────┘                          │
                                                                 ▼
                                                  DataStore, no aparelho
```

## 2. Corrotinas para a interface

A aula de Kotlin apresentou o `suspend`: uma função que pode esperar sem prender a thread. Para a interface, isso resolve um problema concreto. A thread principal desenha a tela e responde aos toques; se ela parar para esperar a rede, a tela congela. Com corrotinas, a espera suspende a corrotina, e a thread continua livre.

| Peça | O que faz |
|---|---|
| `suspend fun` | função que pode esperar (rede, disco); só pode ser chamada de outra `suspend` ou de uma corrotina |
| `launch { }` | inicia uma corrotina e segue em frente |
| `CoroutineScope` | dono de um grupo de corrotinas: quando ele termina, as corrotinas dele são canceladas |
| `viewModelScope` | o escopo do ViewModel, cancelado quando o ViewModel deixa de existir |
| `Dispatchers.Main` | a thread da interface; as corrotinas do `viewModelScope` começam nela |

A regra de que toda corrotina tem um dono se chama concorrência estruturada. Na prática: se o usuário sai da tela antes de a lista chegar, a busca é cancelada, sem continuar gastando rede e sem tentar atualizar uma tela que não existe mais.

O cancelamento é feito com uma exceção, `CancellationException`. Por isso, um `catch` genérico (`catch (e: Exception)`) precisa deixá-la passar, relançando-a; senão, a corrotina continua como se nada tivesse acontecido.

No desktop, o `Dispatchers.Main` vem da biblioteca `kotlinx-coroutines-swing`, acrescentada no Passo 11. No Android, ele já existe.

📖 Ref. Android — Kotlin coroutines on Android: https://developer.android.com/kotlin/coroutines

Uma corrotina não é uma thread: é um objeto leve, agendado sobre poucas threads reais. Criar milhares de corrotinas é barato; criar milhares de threads, não.

| | **Thread** | **Corrotina** |
|---|---|---|
| custo de criar | alto (pilha do SO) | baixo (um objeto) |
| quem agenda | o sistema operacional | o despachante da linguagem |
| troca de contexto | o SO | pontos de suspensão, cooperativo |

O cancelamento também é **cooperativo**: a corrotina só para nos pontos de suspensão — uma função `suspend` que não suspende nunca percebe o cancelamento. Por isso toda função `suspend` deve ser "cancelável": propagar a `CancellationException`, nunca engolir.

📖 Ref. Kotlin — coroutines: https://kotlinlang.org/docs/coroutines-overview.html

## 3. `StateFlow` e o ViewModel

### 3.1 Flow e StateFlow

Um `Flow<T>` é uma sequência de valores que chegam ao longo do tempo. Um `StateFlow<T>` é um `Flow` com uma particularidade: ele sempre tem um valor atual, e emite um valor novo quando o atual muda. É a forma natural de representar o estado de uma tela.

```
private val _estado = MutableStateFlow(EstadoLista())       // só o ViewModel altera
val estado: StateFlow<EstadoLista> = _estado.asStateFlow()   // a tela só lê

_estado.update { it.copy(texto = novo) }
```

- O estado é uma `data class` imutável. Mudar é criar uma cópia com `copy`.
- `update { }` aplica a mudança de forma segura, mesmo com várias corrotinas.
- O `MutableStateFlow` é privado: a tela recebe a versão somente leitura e não consegue alterar o estado por fora.

📖 Ref. Android — StateFlow and SharedFlow: https://developer.android.com/kotlin/flow/stateflow-and-sharedflow

### 3.2 O ViewModel

O ViewModel guarda o estado fora da composição e é onde ficam as decisões: a tela mostra o estado e repassa os eventos.

```
class TarefasViewModel(private val repositorio: RepositorioDeTarefas) : ViewModel() {
    private val _estado = MutableStateFlow(EstadoLista())
    val estado: StateFlow<EstadoLista> = _estado.asStateFlow()

    fun mudarTexto(texto: String) = _estado.update { it.copy(texto = texto) }
    fun adicionar() { ... }
    fun alternar(id: Int) = _estado.update { ... }
}
```

Na tela:

```
val estado by viewModel.estado.collectAsStateWithLifecycle()

TelaLista(estado.tarefas, estado.texto, viewModel::mudarTexto, viewModel::adicionar,
        viewModel::alternar, ...)
```

- `collectAsStateWithLifecycle` lê o `StateFlow` como estado do Compose e para de coletar quando a tela sai de vista.
- `viewModel::adicionar` é uma referência à função, com o tipo `() -> Unit` que a tela espera (capítulo 6 da leitura de Kotlin).
- O ViewModel chega à tela por parâmetro, com um valor padrão. É o que permite aos testes passar um ViewModel montado por eles.

Uma observação de versão: a biblioteca de ViewModel precisa ser da mesma família que a da Navigation (2.9.x). Com versões desalinhadas, a navegação falha com um erro de ciclo de vida ("State must be at least CREATED") ao voltar de uma tela.

📖 Ref. Compose Multiplatform — ViewModel: https://kotlinlang.org/docs/multiplatform/compose-viewmodel.html

## 4. Os estados da tela como classe selada

### 4.1 A classe selada

Uma tela que busca dados tem quatro situações, e a rubrica pede as quatro:

```
sealed interface EstadoTela {
    data object Carregando : EstadoTela
    data object Vazio : EstadoTela
    data class Sucesso(val tarefas: List<Tarefa>, val doCache: Boolean = false) : EstadoTela
    data class Erro(val mensagem: String) : EstadoTela
}
```

Numa classe selada, todos os casos são conhecidos na compilação. Um `when` sobre ela não precisa de `else`, e não deve ter:

```
when (val tela = estado.tela) {
    EstadoTela.Carregando -> TelaCarregando()
    is EstadoTela.Erro -> TelaErro(tela.mensagem, onTentarDeNovo = viewModel::carregar)
    EstadoTela.Vazio, is EstadoTela.Sucesso -> TelaLista(estado.tarefas, ...)
}
```

Se alguém acrescentar um quinto estado, este `when` deixa de compilar até que se decida como mostrá-lo. Com `else`, o caso novo cairia no ramo genérico sem aviso. Dentro do ramo `is EstadoTela.Erro`, `tela` já é tratada como `Erro`, e `tela.mensagem` existe.

`data object` é um `object` com `toString` legível (aparece como `Carregando` num log ou numa falha de teste), útil para os casos sem dados.

📖 Ref. Kotlin — Sealed classes: https://kotlinlang.org/docs/sealed-classes.html

### 4.2 Carregar numa corrotina

```
fun carregar() {
    _estado.update { it.copy(tela = EstadoTela.Carregando) }
    viewModelScope.launch {
        val tela = try {
            val listagem = repositorio.listar()
            if (listagem.tarefas.isEmpty()) EstadoTela.Vazio else
        EstadoTela.Sucesso(listagem.tarefas, listagem.doCache)
        } catch (e: FalhaNoRepositorio.SemConexao) {
            EstadoTela.Erro("Sem conexão com o servidor. Confira a rede e tente de novo.")
        } catch (e: FalhaNoRepositorio.RespostaInvalida) {
            EstadoTela.Erro("O servidor respondeu algo inesperado (${e.message}).")
        }
        _estado.update { it.copy(tela = tela) }
    }
}
```

O ViewModel chama `carregar()` no `init`, e o botão **Tentar de novo** chama de novo. No Passo 12, um `catch (e: Exception)` genérico precisava relançar o `CancellationException`. A partir do Passo 14, os `catch` capturam só as falhas do domínio (capítulo 6): qualquer outra exceção, inclusive o cancelamento, segue adiante sem tratamento especial.

## 5. Camadas e injeção de dependência com Koin

### 5.1 Três camadas

```
br.ufrn.exemplo.tarefas
├── App.kt, Modulos.kt        tema, navegação e a raiz de composição
├── dominio/                  Tarefa, regras, Listagem, a porta RepositorioDeTarefas, as
falhas
├── dados/                    RepositorioRemoto, RepositorioComCache, RepositorioEmMemoria
└── apresentacao/             TarefasViewModel, EstadoTela, as telas
```

| Camada | Pode depender de | Não pode depender de |
|---|---|---|
| domínio | só Kotlin | Compose, Ktor, DataStore, Koin |
| dados | domínio; bibliotecas de rede e de disco | apresentação |
| apresentação | domínio; Compose; ViewModel | detalhes de dados |

É a mesma regra de dependência da API de tarefas, aplicada ao app: as setas apontam para o domínio. O ViewModel conhece a interface `RepositorioDeTarefas`, e não quem a implementa.

### 5.2 Koin

O Koin monta as peças. Toda a ligação fica num lugar só, a raiz de composição:

```
fun moduloTarefas(diretorioDeDados: String) = module {
    single { criarClienteHttp() }
    single { CacheDeTarefas(criarDataStore(diretorioDeDados)) }
    single<RepositorioDeTarefas> {
        val endereco = enderecoDaApi
        if (endereco == null) RepositorioEmMemoria()
        else RepositorioComCache(RepositorioRemoto(get(), endereco), get())
    }
    viewModelOf(::TarefasViewModel)
}
```

- `single` cria uma instância e a reaproveita; `get()` pede ao Koin uma dependência já declarada.
- `viewModelOf(::TarefasViewModel)` cria o ViewModel passando ao construtor o que ele pede.
- Em `App()`, `KoinApplication(application = { modules(...) }) { ... }` disponibiliza o Koin para a interface; em `Conteudo`, `koinViewModel()` pede o ViewModel.
- A escolha da implementação fica num lugar só. Sem endereço de API, o módulo entrega o `RepositorioEmMemoria`: é assim que o app roda sem a API de tarefas (capítulo 6.4).

Nos testes, o Koin não aparece: o ViewModel é criado diretamente, com o dublê que o teste quiser. Um teste separado (`ModulosTest`) confere que o módulo consegue montar o ViewModel.

📖 Ref. Koin — Compose Multiplatform: https://insert-koin.io/docs/reference/koin-compose/compose

## 6. Dados reais com o Ktor Client

### 6.1 DTO separado da entidade

```
@Serializable
data class TarefaDto(val id: Int, val titulo: String, val feita: Boolean = false)

fun TarefaDto.paraDominio() = Tarefa(id = id, titulo = titulo, feita = feita)
```

O DTO (*data transfer object*) tem o formato do JSON da API. A `Tarefa` do domínio tem o formato que o app precisa. Hoje são iguais; separados, uma mudança de nome de campo na API muda o DTO e a

conversão, e o domínio e as telas continuam como estão.

### 6.2 O repositório remoto

```
override suspend fun listar(): Listagem = traduzindoFalhas {
    Listagem(cliente.get("$enderecoDaApi/tarefas").body<List<TarefaDto>>().map {
        it.paraDominio() })
}
```

O cliente HTTP é criado sem motor explícito: cada plataforma usa o que estiver disponível (OkHttp no Android, CIO no desktop). Duas configurações importam:

- `expectSuccess = true`: respostas 4xx e 5xx viram exceção, em vez de chegarem como se estivessem certas;
- `json(Json { ignoreUnknownKeys = true })`: um campo novo na API não quebra o app.

📖 Ref. Ktor — Client: https://ktor.io/docs/client-create-new-application.html

### 6.3 Falhas com nome

```
sealed class FalhaNoRepositorio(mensagem: String) : Exception(mensagem) {
    class SemConexao : FalhaNoRepositorio("sem conexão com o servidor")
    class RespostaInvalida(detalhe: String) : FalhaNoRepositorio("resposta inválida:
        $detalhe")
}
```

| O que aconteceu | Exceção do Ktor | Falha do domínio |
|---|---|---|
| API fora do ar, sem rede, conexão recusada | `IOException` | `SemConexao` |
| status 4xx ou 5xx | `ResponseException` | `RespostaInvalida` |
| JSON fora do formato | `ContentConvertException`, `SerializationException` | `RespostaInvalida` |

O repositório traduz as exceções da biblioteca em falhas do domínio. Assim o ViewModel não conhece o Ktor, e cada falha tem a sua mensagem na tela: é o que a rubrica pede em "falha de conexão e resposta inválida tratadas, cada uma com estado de tela próprio".

Quando salvar uma tarefa falha, a lista continua na tela, e a falha aparece como um aviso acima dela (`EstadoLista.aviso`), em vez de substituir tudo por uma tela de erro.

### 6.4 Onde está a API

```
expect val enderecoDaApi: String?     // commonMain; nulo quer dizer "sem API"


// desktopMain
actual val enderecoDaApi: String? = when (val valor = System.getenv("TAREFAS_API")) {
    null, "" -> "http://localhost:8080"
    "memoria" -> null
    else -> valor
}
// androidMain
actual val enderecoDaApi: String? = "http://10.0.2.2:8080"
```

`expect` declara, no `commonMain`, um valor que cada plataforma define; `actual` é a definição de cada alvo, aqui o desktop e o Android.

Sem a API (no Codespace, no laboratório, ou só para mexer na interface), o app roda com o repositório em memória: `TAREFAS_API=memoria ./gradlew :composeApp:run`, ou `mise run app:sem-api`. O Codespace do repositório já vem com `TAREFAS_API=memoria`. No Android, troca-se o endereço por `null`.

No emulador Android, `10.0.2.2` é o computador que roda o emulador. Num celular de verdade, é o IP do computador na mesma rede Wi-Fi. O Android precisa da permissão `INTERNET` e, para `http://` sem TLS, de `android:usesCleartextTraffic="true"`; num app publicado, a API usa `https`, e essa linha sai.

# 7. Abrir sem rede: cache com DataStore

## 7.1 Um repositório que usa outro

```
class RepositorioComCache(
    private val remoto: RepositorioDeTarefas,
    private val cache: CacheDeTarefas,
) : RepositorioDeTarefas {

    override suspend fun listar(): Listagem = try {
        remoto.listar().also { cache.salvar(it.tarefas) }
    } catch (e: FalhaNoRepositorio.SemConexao) {
        val guardadas = cache.ler() ?: throw e
        Listagem(guardadas, doCache = true)
    }
}
```

Com conexão, guarda cada lista recebida. Sem conexão, devolve a última guardada, marcada com `doCache = true`; se nada foi guardado ainda, o erro continua. O ViewModel não sabe que existe cache: só recebe a marcação, e a tela mostra "Sem conexão: mostrando a última lista salva.". É o critério "Persistência local": o app abre sem rede mostrando o último conteúdo.

Repare que `RepositorioComCache` implementa a mesma interface que recebe. Essa forma, um objeto que envolve outro do mesmo tipo e acrescenta um comportamento, é o padrão Decorator.

## 7.2 O DataStore

O DataStore guarda pares chave e valor num arquivo, de forma assíncrona e segura. O cache do exemplo guarda a lista como o JSON dos DTOs, numa chave só:

```
suspend fun salvar(tarefas: List<Tarefa>) {
    dataStore.edit { it[chave] = Json.encodeToString(tarefas.map { TarefaDto(it.id,
        it.titulo, it.feita) }) }
}
```

Dois cuidados:

- **Um DataStore por arquivo, por processo.** Criar dois para o mesmo arquivo é erro. O Koin cria um só (`single`).
- **Cada plataforma tem a sua pasta.** O diretório vem de fora, como parâmetro de `App`: no Android, `filesDir.path`, a pasta privada do app; no desktop, `~/.tarefas-compose`.

Para listas grandes, buscas e relações entre dados, o caminho é um banco no aparelho (Room ou SQLDelight, os dois com suporte a Kotlin Multiplatform). Para uma lista pequena, o DataStore basta.

📖 Ref. DataStore — Kotlin Multiplatform: https://developer.android.com/kotlin/multiplatform/datastore

## 8. Testes de ViewModel com Turbine

### 8.1 O problema

O ViewModel emite uma sequência de estados: `Carregando`, depois `Sucesso` (ou `Erro`). Um teste que só olha o estado final não vê se o `Carregando` apareceu. E o repositório real depende de rede e de tempo.

### 8.2 Dublê e relógio virtual

```
class RepositorioFalso(var listar: () -> Listagem = { ... }) : RepositorioDeTarefas {
    override suspend fun listar(): Listagem {
        delay(500)                    // "rede", em tempo virtual
        return listar.invoke()
    }
}
```

O dublê responde o que o teste mandar, inclusive lançando `SemConexao` ou `RespostaInvalida`. O `delay` dentro de um `runTest` usa um relógio virtual: os 500 ms passam na hora.

### 8.3 Turbine

```
@Test
fun carregarPassaPorCarregandoEChegaASucesso() = runTest {
    Dispatchers.setMain(StandardTestDispatcher(testScheduler))
    TarefasViewModel(RepositorioFalso()).estado.test {
        assertEquals(EstadoTela.Carregando, awaitItem().tela)
        assertIs<EstadoTela.Sucesso>(awaitItem().tela)
    }
}
```

- `Dispatchers.setMain(StandardTestDispatcher(testScheduler))`: as corrotinas do `viewModelScope` passam a usar o relógio do teste.
- `.test { }` é o Turbine: dentro do bloco, cada `awaitItem()` é o próximo valor emitido, na ordem. O teste vê a sequência, e não só o fim.

- `skipItems(n)` pula estados que não interessam ao teste.

O exemplo tem 7 testes de ViewModel (carregando → sucesso, vazio, sem conexão, resposta inválida, tentar de novo, lista do cache, adicionar) e 7 de interface, que agora também cobrem a tela de erro e o aviso de cache. A rubrica pede ≥ 5 de cada tipo, verdes no CI.

Os testes de interface, que rodam o Compose, também precisam de um `Dispatchers.Main` de teste; ali, `UnconfinedTestDispatcher()`, que executa as corrotinas na hora.

📖 Ref. Turbine: https://github.com/cashapp/turbine

📖 Ref. Android — Testing coroutines: https://developer.android.com/kotlin/coroutines/test

### 8.4 Os outros testes

| Arquivo | O que testa | Como |
|---|---|---|
| `RepositorioRemotoTest` | JSON → tarefas; servidor fora; status 500; JSON fora do formato | `MockEngine` do Ktor, um servidor HTTP de mentira |
| `RepositorioComCacheTest` | guarda; devolve sem conexão; erro sem nada guardado | `RepositorioEmMemoria` no lugar da rede, DataStore num arquivo temporário |
| `ModulosTest` | o módulo do Koin monta o ViewModel | `koinApplication { modules(...) }` |

```
./gradlew :composeApp:desktopTest       # os 25, no alvo desktop, sem emulador
```

## 9. Exercícios e dúvidas frequentes

### Exercícios

1. No projeto do grupo, leve o estado de uma tela para um ViewModel com `StateFlow`.
2. Modele os estados dessa tela como classe selada. Acrescente um quinto estado de propósito e veja o `when` deixar de compilar.
3. No exemplo, troque no `Modulos.kt` o repositório pelo `RepositorioEmMemoria` e rode o app sem a API. O que mudou na tela? E no ViewModel?
4. Com a API de tarefas no ar, rode o app; depois pare a API e toque em **Tentar de novo**. Feche e abra o app. Qual mensagem aparece em cada momento, e por quê?
5. Escreva um teste com Turbine para "adicionar sem conexão mostra um aviso e mantém a lista".
6. Faça a API devolver um campo a mais (por exemplo, `"prioridade": 1`) e confira que o app continua funcionando. Que configuração garante isso?

### Dúvidas frequentes

**Preciso usar a API de tarefas?** Não. A rubrica pede dados do backend escolhido pelo grupo na Sprint 0. O exemplo usa a API de Tarefas porque ela já existe e é conhecida.

**Koin ou injeção manual?** As duas resolvem. O que a rubrica cobra é o efeito: o ViewModel recebe o repositório pelo construtor, por interface, e não o cria.

**DataStore, Room ou SQLDelight?** DataStore para preferências e listas pequenas, guardadas inteiras. Room ou SQLDelight quando houver consultas, relações ou volume.

**Os testes de interface rodam no CI?** Rodam no alvo desktop, sem emulador. No runner Linux do GitHub Actions, instalem antes `libgl1 libegl1 libfontconfig1`, como descrito no guia da Sprint 1.

## Referências

Todas as páginas foram consultadas em outubro de 2026.


Corrotinas, Flow e ViewModel

- Android — Kotlin coroutines on Android: https://developer.android.com/kotlin/coroutines
- Android — StateFlow and SharedFlow: https://developer.android.com/kotlin/flow/stateflow-and-sharedflow
- Compose Multiplatform — ViewModel: https://kotlinlang.org/docs/multiplatform/compose-viewmodel.html
- Kotlin — Sealed classes: https://kotlinlang.org/docs/sealed-classes.html
- Kotlin — Coroutines basics: https://kotlinlang.org/docs/coroutines-basics.html

Camadas, rede e persistência

- Koin — Compose Multiplatform: https://insert-koin.io/docs/reference/koin-compose/compose
- Ktor — Client: https://ktor.io/docs/client-create-new-application.html
- DataStore — Kotlin Multiplatform: https://developer.android.com/kotlin/multiplatform/datastore

Testes

- Turbine: https://github.com/cashapp/turbine
- Android — Testing coroutines: https://developer.android.com/kotlin/coroutines/test